

```

---
title: "Algoritmos"
output:
  pdf_document: default
  word_document: default
  html_document: default
date: "2026-04-01"
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

```

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros comandos de R

R es un lenguaje muy similar a matlab y permite el trabajo con matrices

```

```{r}
A=48
A=40
B=5
C=A+B
C
```

```

```

```{r}
Y=40
alpha<-30
```

```

##Uso de datasets o base de datos internos de R
 ## RMarkdown en Posit Cloud – Guía básica completa

****¿Qué es RMarkdown?****

RMarkdown te permite crear documentos dinámicos que combinan código R, resultados como gráficos y tablas, y texto en prosa. Podés usarlo para análisis interactivos, reproducir resultados, colaborar y comunicar conclusiones, todo desde el navegador sin instalar nada.

****Ventajas de Posit Cloud****

Posit Cloud te da acceso a RStudio completo desde cualquier navegador sin instalar nada, con proyectos aislados que encapsulan código y datos. Tiene plan gratuito con 25 horas mensuales de cómputo y permite compartir proyectos fácilmente con otros usuarios.

****Estructura de un archivo .Rmd****

Todo archivo RMarkdown tiene tres partes. El encabezado YAML va al principio y define el título, autor, fecha y formato de salida como HTML, PDF o Word. Luego viene el texto en Markdown donde escribís el contenido narrativo, títulos y listas. Finalmente los chunks de código, que son bloques donde va el código R que se ejecuta al renderizar.

****Formatos de salida****

Desde el mismo archivo `.Rmd` podés generar reportes en HTML, PDF o Word, notebooks interactivos, presentaciones en PowerPoint o Beamer, dashboards, aplicaciones Shiny y hasta sitios web o blogs.

****Cosas básicas dentro de un chunk****

R trae datasets internos listos para usar como ``cars``, ``HairEyeColor`` o ``iris``, que cargás con ``data(nombre)`` y explorás con funciones como ``head()`` o ``summary()``. El operador ``$`` te permite acceder a una columna específica de un data frame, por ejemplo ``cars$speed``, y combinarlo con funciones como ``mean()`` o ``sum()``. También podés crear variables con ``<-`` y operar entre ellas, como ``a <- 48``, ``b <- 3``, ``c <- a + b``, usando suma, resta, multiplicación, división o potencia.

****Gráficos con plot()****

La función principal para graficar en R es ``plot()``, que acepta uno o dos vectores de datos y genera un gráfico de dispersión por defecto. Para personalizar el gráfico usás argumentos dentro de la misma función: ``main`` define el título principal, ``xlab`` y ``ylab`` cambian las etiquetas de los ejes, ``col`` cambia el color de los puntos o líneas, ``type`` define el tipo de gráfico donde ``"p"`` es puntos, ``"l"`` es líneas y ``"b"`` es ambos, y ``pch`` cambia la forma de los puntos. Por ejemplo, ``plot(cars$speed, cars$dist, main = "Velocidad vs Distancia", xlab = "Velocidad", ylab = "Distancia", col = "blue", type = "p")`` genera un gráfico completo con título y ejes etiquetados.

Además de ``plot()``, R tiene funciones específicas para otros tipos de gráficos. ``hist()`` genera histogramas de frecuencia, ``barplot()`` crea gráficos de barras, ``boxplot()`` muestra la distribución de los datos con cuartiles y ``pie()`` genera gráficos de torta. Todos estos aceptan los mismos argumentos de personalización como ``main``, ``xlab``, ``ylab`` y ``col``.

Para agregar elementos sobre un gráfico ya existente podés usar ``lines()`` para añadir una línea, ``points()`` para sumar puntos adicionales, ``legend()`` para insertar una leyenda y ``abline()`` para trazar líneas de referencia como una media o una recta de regresión.

****Opciones de los chunks****

Al configurar un chunk controlás qué aparece en el documento final. Con ``echo = FALSE`` ocultas el código pero mostrás el resultado, con ``include = FALSE`` ejecutás en silencio sin mostrar nada, y con ``message = FALSE`` suprimís los mensajes que genera R al cargar paquetes.

****Otros lenguajes y reproducibilidad****

Además de R, Posit Cloud permite correr chunks en Python, Bash, SQL, Julia y Stan dentro del mismo documento. Al tener código y análisis juntos, los resultados se reproducen exactamente al volver a ejecutar el archivo, y los informes pueden actualizarse automáticamente cuando cambian los datos.

Funciones estadísticas

```
```{r}
z1<-rnorm(350,21,5)
z1
```
```

```
```{r}
plot(z1)
```
```

```
```{r}
hist(z1,main="Histograma de edades")
```
```

```
```{r}
density(z1)
```
```

```
```{r}
plot(density(z1))
```
```

You can also embed plots, for example:

```
```{r cars}
plot(pressure)
```
```

```
```{r pressure}
plot(pressure,main="presion del gas ideal",ylab="hPa",xlab="K",type="l",
col="yellow")
```
```

Consigna 1: Las últimas 3 cifras de mi DNI son 364, crear una variable que tenga ese número.

```
```{r}
DNI=364
```
```

Consigna 2: Crear un vector que tenga todos los números del 1 al 364 enteros.

```
```{r}
for(i in 1:364)
 print(1:i)
```

```{r}
secuenciaDNI <- c()
for(i in 1:364){
 secuenciaDNI <- c(secuenciaDNI, i)
}
```
```

Consigna 3: Calcula la suma de todos los valores del vector secuencia DNI con R.

```
```{r}
total<-0
valorfinal<-length(secuenciaDNI)
for (i in 1:valorfinal){
 total<-total+secuenciaDNI[i]
}
print(total)
```
```

Consigna 4: Calcula la suma de todos los valores del vector secuenciaDNI con python.

```
```{python}
secuenciaDNI = range(1, 365) # incluye 364
total = sum(secuenciaDNI)
print(total)
```
```

Consigna 5: Calcula la suma de todos los valores del vector secuenciaDNI con Julia. No se programó para no ocupar memoria.

Consigna 6: Determinar CPU time used.

```
```{r}
require(stats)
system.time(for(i in 1:100) mad(runif(1000)))
```

```{r}
require(stats)
system.time(for(i in 1:length(secuenciaDNI)){})
```

```{r}
total<-0
valorfinal<-length(secuenciaDNI)
for (i in 1:valorfinal){
 total<-total+secuenciaDNI[i]
}
print(total)
```

```{r}
sleep_for_a_minute <- function() { Sys.sleep(14) }
start_time <- Sys.time()
sleep_for_a_minute()
end_time <- Sys.time()
end_time- start_time
```
```

```
```
```

Consigna 7: Comparación de performance con `system.time()`.

```
```{r}
```

```
system.time({
  A <- numeric(50000)
  for (i in 1:50000) {
    A[i] <- i * 2
  }
})
system.time({
  B <- seq(from = 2, to = 100000, by = 2)
})
```
```

Biblioteca `tictoc`

Esto de usar una biblioteca es llamar u cargar una procedimientos que generará comando nuevos en R. Como ya fue comentado, cargar una biblioteca implica ejecutar

el comando `install.packages()` o usar en r-studio el menú de Herramientas y Luego Instalar paquetes. Las funciones `tic` y `toc` son de la misma biblioteca de Octave/Matlab y se usan de la misma manera para la evaluacion comparativa que el tiempo de sistema recién demostrado. Sin embargo, `tictoc` agrega mucha más comodidad al usuario y armonía al conjunto.

```
```{r}
```

```
library(tictoc)
tic("sleeping")
A<-20
print("dormire una siestita...")
## [1] "dormire una siestita..."
Sys.sleep(2)
print("...suena el despertador")
## [1] "...suena el despertador"
toc()
```

```
```
```

Biblioteca `rbenchmark` La documentación de la función `benchmark` del paquete `rbenchmark` R lo describe como un simple contenedor alrededor de `system.time`. Sin

embargo, agrega mucha conveniencia en comparación con las llamadas simples a `system.time`. Por ejemplo, requiere solo una llamada de referencia para cronometrar

múltiples repeticiones de múltiples expresiones. Además, los resultados devueltos se organizan convenientemente en un marco de datos.

Implementación de una serie Fibonacci o Fibonacci

En matemáticas, la sucesión o serie de Fibonacci es la siguiente sucesión infinita de números naturales: 0, 1, 1, 2, 3, 5, 8 ... 89, 144, 233 ...

La sucesión comienza con los números 0 y 1, 2 a partir de estos, cada término es la suma de los dos anteriores , es la relación de recurrencia que la define.

A los elementos de esta sucesión se les llama números de Fibonacci. Esta sucesión fue descrita en Europa por Leonardo de Pisa, matemático italiano del siglo XIII

también conocido como Fibonacci. Tiene numerosas aplicaciones en ciencias de la computación, matemática y teoría de juegos. Original de la Biblioteca

Universidad

de Florencia. Liber Abachi- Autor Fibonacci

Consigna 8: ¿Cuántas iteraciones se necesitan para generar un número de la

serie mayor que 1.000.000 ?

```
```{r}
```

```
# Inicialización
```

```
a <- 0
```

```
b <- 1
```

```
contador <- 1 # ya tenemos el primer número
```

```
# Iterar hasta superar 1.000.000
```

```
while (b <= 1000000) {
```

```
  siguiente <- a + b
```

```
  a <- b
```

```
  b <- siguiente
```

```
  contador <- contador + 1
```

```
}
```

```
# Resultados
```

```
cat("Número de iteraciones:", contador, "\n")
```

```
cat("Primer Fibonacci mayor a 1.000.000:", b, "\n")
```

```
```
```

Compara la performance de ordenación del método burbuja vs el método sort de R

Usar método microbenchmark para una muestra de tamaño 20.000

```
```{r}
```

```
# Instalar paquete si no lo tenés
```

```
if (!require(microbenchmark)) {
```

```
  install.packages("microbenchmark")
```

```
  library(microbenchmark)
```

```
}
```

```
# Generar muestra
```

```
set.seed(123)
```

```
n <- 20000
```

```
vector <- sample(1:100000, n, replace = TRUE)
```

```
# Implementación de ordenamiento burbuja
```

```
bubble_sort <- function(x) {
```

```
  n <- length(x)
```

```
  for (i in 1:(n-1)) {
```

```
    for (j in 1:(n-i)) {
```

```
      if (x[j] > x[j+1]) {
```

```
        temp <- x[j]
```

```
        x[j] <- x[j+1]
```

```
        x[j+1] <- temp
```

```
      }
```

```
    }
```

```
}
```

```
  return(x)
```

```
}
```

```
# Benchmark (OJO: burbuja es muy lento)
```

```
resultado <- microbenchmark(
```

```
  burbuja = bubble_sort(vector),
```

```
  sort_R = sort(vector),
```

```
  times = 5
```

```
)
```

```
print(resultado)
```

```
```
```

La penitencia de Newton

Comentan algunos historiadores que Newton tenía en la escuela primaria un

profesor muy exigente que reclamaba constantemente la atención de los alumnos a lo que

el estaba enseñando. Tan pronto alguien cometía un acto improcedente lo castigaba con una penitencia que consistía en traer para el día siguiente el resultado de la suma desde el número 1 hasta cierto número aleatoriamente elegido. Por supuesto esta tarea demandaría quedarse toda la noche haciendo las sumas parciales, tarea que el profesor ya había realizado previamente y que tenía registrado en un voluminoso cuaderno que (al estilo tabla de logaritmos) cargaba religiosamente en su portafolios de cuero.

Consigna 9: Desarrolla dos algoritmos que hagan este trabajo, por ejemplo para sumar desde 1 hasta  $10^{10}$  y verifica cuál de los dos es más eficiente.

```
```\r}
suma_iterativa <- function(n) {
  total <- 0
  for (i in 1:n) {
    total <- total + i
  }
  return(total)
}
suma_formula <- function(n) {
  return(n * (n + 1) / 2)
}
```
```

Se desarrollaron dos algoritmos para calcular la suma de los primeros  $n$  números naturales. El método iterativo presenta complejidad lineal, lo que lo hace ineficiente para valores grandes como  $10^{10}$ . En cambio, el uso de la fórmula cerrada reduce la complejidad a constante, permitiendo obtener el resultado de forma inmediata. Por lo tanto, el segundo algoritmo es significativamente más eficiente.